

DSA STUDY BLUEPRINT SERIES

56. Doubly Linked List Tutorial

Double Linked List Pointer Re-wirings

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **DLL:** Nodes contain double pointers: `prev` and `next`.
- Allows bidirectional list traversal.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
struct DLLNode {  
    int val;  
    DLLNode *prev, *next;  
    DLLNode(int x) : val(x), prev(nullptr), next(nullptr) {}  
};  
// Reverse DLL: swap 'next' and 'prev' pointers for all nodes
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Reversal / Traversal	$O(N)$ time, $O(1)$ space
Deletion (Target Node)	$O(1)$ (Direct pointer)

Key Practices & Gotchas

- **Important:** Ensure update of boundary node pointers when inserting or deleting elements.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment 10 → 20:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	curr->next = prev	10 points to nullptr
2	[10, 20, nullptr]	curr->next = prev	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.